HN Breakout: Early Prediction of Hacker News Front-Page Posts

MLOps HS26 – MS2 Summary (Feature Pipeline)

David Rivera Allegro

Repository: github.com/AbsturzWindows/hn-breakout (public)

5 November 2026

1 What I achieved in this milestone

- **The poller runs on its own.** Since 13.10.2026, 18:04 UTC, a Cloud Run Job runs every 5 minutes, started by Cloud Scheduler. Each run saves a snapshot of every story younger than 35 minutes, the current top 30 of the front page, the label of every story that just turned 24 hours old, and one row about the run itself (counts, errors, duration). Everything is stored as Parquet on Google Cloud Storage, partitioned by day (`{table}/date=YYYY-MM-DD/`). This folder structure is my feature store.
- **Features are built from the snapshots.** A second job runs once a day at 03:15 UTC and turns the snapshots of the last three days into one row per labelled story (`features` table). All feature definitions live in one module that training and inference both import, so both compute them the same way: `src/hn_breakout/features.py`. Since labels arrive 24 hours after posting, each day is rebuilt on three nights and is complete after the second. Every rebuild overwrites the same file, so there are no duplicate rows, and training only reads days that are at least two days old.
- **Data completeness.** `scripts/completeness.py` reports per day how many runs ran, failed or were missed, and how many stories have all three momentum checkpoints (10/20/30 min). Up to 02.11, every day is fully labelled (full table: `docs/completeness.txt`):

Day	Runs	Failed	Stories	Complete	Hits
13.10 (from 18:04)	71	0	291	97.6 %	2.1 %
14.10	288	0	1211	99.2 %	3.6 %
15.10	288	0	1250	98.6 %	2.6 %
16.10	288	0	1125	99.1 %	3.0 %
17.10 (Sat)	288	0	764	99.0 %	3.4 %
18.10 (Sun)	288	0	703	98.2 %	4.6 %
19.10	288	0	1123	98.3 %	3.6 %
20.10	287	0	1309	98.6 %	2.9 %
21.10	288	1	1157	96.5 %	3.6 %
22.10	288	0	1241	98.9 %	3.1 %
23.10	288	0	1110	99.0 %	3.1 %
24.10 (Sat)	288	0	756	98.7 %	4.4 %
25.10 (Sun)	288	0	728	98.9 %	4.1 %
26.10	288	0	1096	99.2 %	3.7 %
27.10	288	0	1131	98.9 %	3.3 %
28.10	288	0	1161	98.5 %	2.3 %
29.10	288	0	1217	99.3 %	3.6 %
30.10	288	0	1025	98.9 %	3.9 %
31.10 (Sat)	288	0	699	99.0 %	3.6 %
01.11 (Sun)	288	0	747	99.2 %	2.9 %
02.11	288	0	1089	99.4 %	4.4 %

In 21 days, 5,830 runs wrote 20,933 labelled stories with about 710 hits (3.4 %). One run is missing (20.10) and one failed (21.10, see below); up to 04.11, no other run failed or was missed. Every story got a label, and 96.5–99.4 % have all three checkpoints. Weekends show roughly 40 % fewer stories than weekdays and, on average, a higher hit rate.

Key decisions.

- *Cloud Scheduler instead of GitHub Actions cron:* the whole dataset depends on a run every 5 minutes, and GitHub cron jobs often start late or not at all.
- *Each step of a run is isolated:* on 21.10 at 15:10 the HN API did not answer `newstories` and `topstories`. That run still wrote its labels and logged the error; only one snapshot was lost. The next runs needed up to 156 s because of retries, still below the 240 s timeout. Normal runs take 50–60 s.
- *Labels catch up on their own:* a label is taken in the window 24–26 hours after submission, so a missed run near the 24-hour mark does not lose the story.
- *Hit threshold fixed before training:* the Algolia estimate gives 4.29 % hits at 100 points, above my 2 % rule, so the label stays at ≥ 100 points (`docs/decisions.md`). For the same days (14.–28.10), Algolia lists 15,573 stories with 661 hits (4.24 %). My poller sees 1.9 % more stories but 20 % fewer hits (530, 3.34 %). Matched by story ID, all 530 of my hits are hits on Algolia too. Of the other 131, 96 had fewer than 10 points after 24 hours and only gained their points days later, most likely through HN's second-chance pool; 16 had 80–99 points, 16 had 10–79 points, and 3 have no label. My label counts points within 24 hours, so these are correctly negative. Even my own 3.34 % is above the 2 % rule, so the decision holds.
- *100 points as a proxy for the front page:* the label is a points threshold, while the alerts are about the front page. Every run saves the top 30, so I can check how well the two agree: 98.9 % of the hits (524 of 530) appear in at least one of these 5-minute top-30 snapshots, but only 39 % of the stories that reached the top 30 also reached 100 points. So a hit is almost always a front-page story, but many front-page stories stay below 100. The label therefore stands for a *strong* front-page story, which is exactly what the alerts should catch.
- **Backfill.** The momentum features cannot be backfilled, because HN keeps no score history (see E2). Two other things can be rebuilt: `hn-build-features START END` rebuilds the features for any date range from the raw snapshots, for

example after a feature change, and `hn-backfill START END` loads past stories from Algolia (title, url, author, final points) into a `history` table, so `domain_hit_rate` has history from the first training day. In MS3 it will only count stories whose label was known before the prediction time. Both can be re-run safely, and a stopped backfill resumes where it stopped (`src/hn_breakout/backfill.py`). The `history` table covers the 90 days before the poller started (15.07.–12.10.2026).

- **What cost me time.** The first deployment failed twice: Cloud Build's default service account had no permissions in my new project, and `gcsfs` needed bucket metadata access just to "create folders", which buckets don't have. I fixed the second one in the code (`storage.py`, commit `4b6cd45`), so the job only needs object permissions. Both fixes are documented in `docs/deploy_gcp.md`. The data starts on 13.10, so by MS3 I will have about seven weeks: five for training and two for testing, one week less than planned in the proposal.
- **Reading order.** Start with `docs/completeness.txt` (whether the data is there) and `docs/deploy_gcp.md` (the cloud setup). Then read `src/hn_breakout/poll.py` and `src/hn_breakout/features.py` for the poller and the features. `uv run pytest` runs 17 tests against a fake HN API and Algolia, and CI runs them together with ruff on every push.

2 Response to MS1 reviews

E1 · Existing HN upvote predictor — Changed (`README.md`, `13acd6f`). Thanks, I had only checked the 2026 showcase. The KTH 2023/24 project (`group-5-id2223/project`) regresses a story's score from its title (BERT embedding), domain and posting time. Its daily pipeline stores the current top ten stories, and its backfill samples random old items, so every story is saved once, with the score it has whenever it is fetched. HN Breakout instead predicts, 30 minutes after posting, whether a story reaches ≥ 100 points within 24 hours. It learns from the story's own vote trajectory, which my poller records every 5 minutes and which no API provides, and the label is always taken at the same age. The model also has to beat a momentum baseline. Their title embedding is the natural next step if A1 shows that momentum alone is not enough.

E2 · The poller is the project — Changed (`scripts/completeness.py`, `docs/completeness.txt`). Agreed, and this is why I deployed the poller before anything else. Every run writes a `runs` row, the completeness script reports missed runs, errors and incomplete stories per day (table above), and a log-based alert in Cloud Monitoring sends a notification to my phone when a run fails.

A1 · Model and baseline share the same signal — Not yet (planned for MS3). Agreed. Reviewer A suggested this for MS2, but training starts in MS3, where it will be the first experiment: a momentum-only model (points at 10/20/30 min, gain, comments) next to the full model, both logged in MLflow. If the full model adds less than 0.03 PR-AUC, I bring the title TF-IDF forward from the stretch goals.

B1 · Too few hits to test the success criterion — Partly. The real numbers are better than expected: with about 1,030 stories per day and 3.4 % hits, two test weeks hold about 490 hits instead of 50–60 (Algolia: 988 stories per day, 4.29 %). I have not run the bootstrap simulation for +0.10 PR-AUC yet. I will run it before the MS3 evaluation (by 03.12) and extend the test window if the confidence interval is wider than 0.10.

B2 · Define the daily top 10 alerts — Changed (`docs/decisions.md`, `1bc5798`). For evaluation: per UTC calendar day, all stories posted that day are ranked by their predicted probability at 30 minutes. I take the top 10 after the day has ended and average the precision over the test days. Live, the day is not over yet, so the UI uses a fixed probability threshold, chosen on the training data so that about 10 stories per day are above it.

E3 · Define the label exactly as computed — Changed (`README.md`, `1bc5798`). The label is the score at the first successful poller run at least 24 hours after submission, usually 24h00–24h05. If no run succeeds between 24 and 26 hours, the story gets no label and is left out. So far this has never happened. "Within 24 hours" in this summary always means this first run after 24 hours. I checked one 2.4-hour window against Algolia: 179 stories in my labels, 176 on Algolia, and the same single hit, with 540 points in my label and 543 on Algolia two days later.

A2 · Snapshot timing — Partly. "Points at 10 min" is the last snapshot at or before 10 minutes, and at most 7 minutes before it, otherwise the value is missing. Measured on all stories: the snapshot is on average 2.5 minutes early (median 2.5, max 7.0), the same for 10, 20 and 30 minutes. I don't adjust to the nominal age: training and inference use the same function, so the deviation is identical in both, and interpolating would need a later snapshot that does not exist yet at prediction time. The deviation only depends on when a story was posted within the 5-minute cycle, so I expect it to be largely independent of whether a story becomes a hit (failed runs are the exception and are rare). If it turns out to matter, I can add the snapshot age as a feature in MS3.

E4 · Record the threshold decision — Changed (`docs/decisions.md`, `0139d22` from 14.10.). Decided and committed on 14.10, before any training: 4.29 % $>$ 2 %, so the threshold stays at 100 points.

E5 · Fallback for the 5-minute jobs — Partly. Only the poller runs every 5 minutes; the feature job runs once a day. The cloud setup was finished early and has run unattended since 13.10 for a few cents per day. There is no automatic failover, but the same Docker image runs locally with `STORAGE_ROOT=./data`, so I can move the poller to another machine in minutes.